Moodleの拡張機能を作ってみよう！｜完全スライド対応ハンズオンガイド

AppLii 勉強会・参加者用（2026年度 Moodle / スライド全107ページ対応）

スライドで説明を聞きながら、ここに載せた手順とコードを自分のPCで実行するための教材です。**スライド番号は最新の `moodle(1).pdf` に合わせています。** 開催は2日を想定していますが、日ごとの区切りは固定しません。

この資料の読み方：①今映っているスライド番号を確認 → ②同じ `p○○` の見出しを探す → ③「どこで実行するか」を確認 → ④コードを入力・保存 → ⑤動作確認。わからなくなったら、章の最後にある「復帰ポイント」を利用してください。

> **安全上の約束**：大学IDのパスワード・ワンタイムコード・MoodleのCookie等は、この拡張機能に入力・保存しません。Microsoftの認証と多要素認証は通常の画面で本人が行います。サンプルは画面遷移の実機検証を保証するものではありません。

スタートする前に（p1〜5）

スライド	スライド内容	参加者がすること
p1〜3	今日作るもの・日々の「めんどくさい」	完成例を見て、何を便利にしたいか考える
p4	事前準備	Chrome・VS Codeを使用できることを確認する
p5	認証の仕組み	Moodle → ログイン → 大学ID認証 → ログイン完了の流れを確認する

作るものは、Moodleの「ログイン」と「和歌山大学IDでログインする」の**2つのリンクのクリック操作**を省く拡張機能です。認証そのものを代行するものではありません。

用意するもの：Chrome、VS Code、Moodleのアカウント。Chromeのプロファイルは自分のものを使ってください。

作業する場所の見分け方

表示	入力する場所
コマンドプロンプト	Windowsのコマンドプロンプト（ターミナル）
VS Code：`manifest.json` など	ファイルを作成／編集して Ctrl+S で保存
Chrome：Console	F12 → Console に入力（ファイルは変更しない）
Chrome：拡張機能	`chrome://extensions/` で更新・有効化

第1部　初めてのChrome拡張機能

対応：スライド p6〜p21　｜　**ここまでの目標：** 自作のJavaScriptがMoodle上で実行される

p6〜8｜考え方　Chrome拡張機能の `content_scripts` を使うと、指定したWebページで自分のJavaScriptを動かせます。

p09〜10｜作業用フォルダーを作る　コマンドプロンプト → VS Code

```
mkdir wakayama-moodle-auto-login
cd wakayama-moodle-auto-login
code .
```

VS Codeが開いたら、左のエクスプローラーで **`manifest.json`** と **`content.js`** を新規作成します。拡張子を `.txt` にしないように注意。

p11〜12｜manifest.jsonを作る　VS Codeで新規作成

Chromeに拡張機能の名前・実行するJavaScript・対象URLを教えるファイルです。

参照ファイル： `code/01-p020-first-extension/manifest.json`

```
{
  "manifest_version": 3,
  "name": "Wakayama Moodle Auto Login",
  "version": "0.1.0",
  "description": "和歌山大学Moodleのログイン操作を自動化する拡張機能",
  "content_scripts": [
    {
      "matches": [
        "https://moodle2026.wakayama-u.ac.jp/2026/*"
      ],
      "js": [
        "content.js"
      ],
      "run_at": "document_idle"
    }
  ]
}
```

p13〜15｜意味を確認　`manifest_version` は仕様バージョン、`name` は名前、`content_scripts` は実行するスクリプト、`matches` は対象URL、`js` はJavaScriptファイル、`run_at` は実行時期です。

p16〜17｜content.jsを作る　VS Codeで新規作成

今回はまだクリックしません。まずConsoleに表示するだけです。

参照ファイル： `code/01-p020-first-extension/content.js`

```
console.log("Moodle拡張機能が動作しました！");
console.log("現在のURL:", window.location.href);
```

`console.log()` は情報をConsoleに表示し、`window.location.href` は現在のURLを取得します。

p18〜20｜Chromeに読み込ませて確認　Chromeの拡張機能画面／Console

1. アドレスバーに `chrome://extensions/` → 右上の「デベロッパーモード」をオン。
2. 「パッケージ化されていない拡張機能を読み込む」→ **自分の作業フォルダー**を選択。
3. Moodleを開く → `F12` → `Console` → `Ctrl + R` で再読み込み。

```
Moodle拡張機能が動作しました！
現在のURL: https://moodle2026.wakayama-u.ac.jp/2026/
```

p21｜理解度チェック　`matches` は何を指定する？ → 答えは「JavaScriptを実行するWebサイトのURL」。

復帰ポイント： `code/01-p020-first-extension/`（p20時点の完成フォルダー）

第2部　Moodleのログインボタンを探す

対応：スライド p22〜p33　｜　**ここまでの目標：** HTMLの要素を取得してConsoleからクリックできる

p23〜25｜HTMLとリンクの仕組み　説明を読む

```
<a href="https://example.com">ここをクリック</a>
```

`<a>` はリンクのタグ、`href` は移動先、タグの間の文字は画面に表示されるテキストです。Moodleのリンクも同じ仕組みです。

```
<a href="https://moodle2026.wakayama-u.ac.jp/2026/login/index.php">
  ログイン
</a>
```

p26｜開発者ツールでログインリンクを調べる　Chrome → 検証

未ログインのMoodleトップで「ログイン」を右クリック →「検証」→ Elementsタブで `<a>` タグと `href` を確認します。すでにログイン済みなら、実験前にMoodleからログアウトします。

p27〜28｜JavaScriptでリンクを取得する　Chrome → Console

Consoleで実行するコードです。VS Codeの `content.js` にはまだ書きません。

参照ファイル： `snippets/02-p027-find-login.js`

```
// スライド27：Moodleの未ログインのトップページでConsoleに貼り付ける
const loginButton = document.querySelector(
  'a[href$="/login/index.php"]'
);
console.log(loginButton);
```

`const` は変数の宣言、`document.querySelector()` は最初に一致したHTML要素の取得、`[href$="..."]` はリンク先の末尾で探す指定です。

p29｜取得したリンクをクリックする　Chrome → Console

p27を実行して `loginButton` が取得できたあと、**同じConsole**で実行します。

参照ファイル： `snippets/02-p029-click-login.js`

```
// スライド29：スライド27のコードに続けてConsoleで実行
loginButton.click();
```

成功するとログイン画面へ移動します。ページが変わるとConsoleの変数は使えなくなるので、必要なら前のページに戻ってp27からやり直します。

p30〜32｜要素がない場合の処理を追加する　Chrome → Console

ログイン済みだと `querySelector()` の結果は `null` になる場合があります。ボタンの有無を `if` で確認してからクリックします。

参照ファイル： `snippets/02-p031-null-check.js`

```
// スライド31：ボタンがなければクリックしない
const loginButton = document.querySelector(
  'a[href$="/login/index.php"]'
);
if (loginButton) {
  console.log("ログインボタンを発見！");
  loginButton.click();
} else {
  console.log("ログインボタンがありません");
}
```

p33｜理解度チェック： `querySelector()` は要素を探す／`.click()` は要素をクリックする。

復帰ポイント： この部はConsoleのみ。 `snippets/` からp27・29・31のコードをコピーできます。

第3部　ログインボタンを自動クリックしよう

対応：スライド p34〜p46　｜　**ここまでの目標：** トップページを開くだけでログイン画面へ進む

p34〜35　いまはConsoleへの手入力が必要です。そのコードを `content.js` に移して自動実行させます。

p36〜38｜content.jsを書き換える　VS Code → content.jsを上書き

上書きしてください。 ここからはMoodleを開いたときにJavaScriptが実行されます。

参照ファイル： `code/03-p036-auto-click/content.js`

```
console.log("Moodle Auto Login 起動！");

const loginButton = document.querySelector(
  'a[href$="/login/index.php"]'
);

if (loginButton) {
  console.log("ログインボタンを発見！");
  loginButton.click();
} else {
  console.log("ログインボタンが見つかりません");
}
```

p39〜40｜拡張機能を更新する　Chrome → 拡張機能

①Ctrl+Sで保存 → ② `chrome://extensions/` で更新 → ③Moodleをログアウト → ④Moodleを開く。ログイン済みならログインリンクがなく、クリックは起きません。

p41〜42｜現在のURLのパスを調べる　Chrome → Console

URLのうち `/2026/` のような部分を `window.location.pathname` から取得します。

参照ファイル： `snippets/03-p042-show-path.js`

```
// スライド42：現在のURLのパスを表示
console.log(window.location.pathname);
```

p43〜44｜トップページだけでクリックする　VS Code → content.jsを上書き

今度は `path === "/2026/"` のときだけクリックする形に改良します。

参照ファイル： `code/03-p043-path-check/content.js`

```
console.log("Moodle Auto Login 起動！");

// Moodleのトップページか確認
if (window.location.pathname === "/2026/") {
  // ログインボタンを探す
  const loginButton = document.querySelector(
    'a[href$="/login/index.php"]'
  );

  // ログインボタンがある場合だけクリック
  if (loginButton) {
    console.log("ログイン画面へ移動します");
    loginButton.click();
  } else {
    console.log("ログインボタンがありません");
  }
}
```

p45〜46｜デバッグと理解度チェック　Chrome → Console

Consoleの「Preserve log（ログを保持）」を有効にすると画面遷移前のログも確認できます。
`window.location.pathname` はURLのパスを返し、`if(loginButton)` は要素なしでのエラーを防ぎます。

復帰ポイント： `p36版` / `p43版` 。前の拡張機能が残っていると二重動作するため、**同時に有効にするのは1つ**にしてください。

第4部　2つ目のログイン画面も自動化しよう！

対応：スライド p47〜p60　｜　**ここまでの目標：** 2つのログインリンクをページごとにクリックする

p47〜48　最初のクリックの次に、大学IDのログインリンクを押す必要があります。

p49｜大学IDログインのHTML　画面を調べる

```
<a href="https://moodle2026.wakayama-u.ac.jp/2026/auth/oidc/">
  和歌山大学IDでログインする
</a>
```

p50｜2つ目のリンクを取得する　Chrome → 2画面目のConsole

リンクが見つかれば `universityButton` にHTMLの要素が入ります。

参照ファイル： `snippets/04-p050-find-oidc.js`

```
// スライド50：大学IDログイン画面のConsoleで実行
const universityButton = document.querySelector(
  'a[href$="/auth/oidc/"]'
);
console.log(universityButton);
```

p51｜大学IDログインへ進む　Chrome → 同じConsole

Microsoftの認証へ進む場合があります。認証コードなどは自動入力せず、正規の画面で操作します。

参照ファイル： `snippets/04-p051-click-oidc.js`

```
// スライド51：スライド50に続けて実行
universityButton.click();
```

p52〜54｜else ifを理解する　Consoleで練習

`if` の条件に当てはまらないときに `else if` を調べます。スライドの例は「数字は2です」と表示されます。

参照ファイル： `snippets/04-p053-if-elseif.js`

```
// スライド53：条件分岐のお試し
const number = 2;
if (number === 1) {
  console.log("数字は1です");
} else if (number === 2) {
  console.log("数字は2です");
} else {
  console.log("1でも2でもありません");
}
```

p55〜57｜2画面に応じて自動クリックする　VS Code → content.jsを上書き

ここは**スライドp55のコードに忠実**な版です。トップページと2つ目の画面で処理を切り替えます。

参照ファイル： `code/04-p055-two-buttons/content.js`

```
console.log("Moodle Auto Login 起動！");

// 今開いているページのパスを取得
const path = window.location.pathname;

// ① Moodleのトップページ
if (path === "/2026/") {
  const loginButton = document.querySelector(
    'a[href$="/login/index.php"]'
  );
  if (loginButton) {
    console.log("ログイン画面へ移動します");
    loginButton.click();
  }
// ② 大学IDログイン画面
} else if (path === "/2026/login/index_form.html") {
  const universityButton = document.querySelector(
    'a[href$="/auth/oidc/"]'
  );
  if (universityButton) {
    console.log("大学ID認証へ移動します");
    universityButton.click();
  }
// ③ それ以外のページ
} else {
  console.log("自動クリックの対象外です");
}
```

p55で動かない場合の確認ポイント：スライドの2画面目の判定は `/2026/login/index_form.html` ですが、最初のログインリンク先は `/2026/login/index.php` です。実際のURLがどちらなのか、Chromeのアドレスバー／Consoleで確かめてください。**第5部p73では2つのパスを両方判定するコード**に変わります。主教材はスライドのまま残しています。

p58〜59｜最初から動作確認　Chrome

Ctrl+S → 拡張機能更新 → Moodleでログアウト → トップから開始。「ログイン」→「大学IDログイン」→認証画面という流れを確認します。

p60｜応用問題　理解度チェック

URLが `/2026/my/` だったら？ → p55の2つの条件には当てはまらないので、最後の `else` が実行されます。

復帰ポイント： `p55時点の完成コード` 。

第5部　安全で安定した拡張機能を作ろう！

対応：スライド p61〜p79　｜　**ここまでの目標：** 自動クリックの繰り返しを制限する

p61〜64　認証に失敗してページが戻った場合、同じクリックを繰り返す可能性があります。クリック済みの状態を一時保存する方針です。

p65｜データを保存する　Chrome → Console

参照ファイル： `snippets/05-p065-save.js`

```
// スライド65：MoodleのConsoleで実行
sessionStorage.setItem("message", "hello");
```

p66｜保存したデータを読む　Chrome → 同じConsole

期待される表示：`hello`

参照ファイル： `snippets/05-p066-read.js`

```
// スライド66
console.log(sessionStorage.getItem("message"));
```

p67〜68｜データを削除する　Chrome → 同じConsole

期待される表示：`null`。`sessionStorage` は基本的にタブ単位で保存され、再読み込み後も残ります。

参照ファイル： `snippets/05-p067-delete.js`

```
// スライド67
sessionStorage.removeItem("message");
console.log(sessionStorage.getItem("message"));
```

p69〜71｜実行済みを判断する　Chrome → Console

1回目は「初めてのクリックです」、同じタブで2回目は「すでにクリック済みです」。`!==` は「等しくない」です。

参照ファイル： `snippets/05-p069-click-guard.js`

```
// スライド69：初回と2回目の違いを試す
const clicked = sessionStorage.getItem("clicked");
if (clicked !== "yes") {
  console.log("初めてのクリックです");
  sessionStorage.setItem("clicked", "yes");
} else {
  console.log("すでにクリック済みです");
}
```

クリック後にページが移動してもよいように、**クリックする前にクリック済み情報を保存**します。

p72〜73｜content.jsを安定化する　VS Code → content.jsを上書き

2つのパスを `||` で判定し、`sessionStorage` に実行済みを記録します。

参照ファイル： `code/05-p073-retry-guard/content.js`

```
console.log("Moodle Auto Login 起動！");
const path = window.location.pathname;

// クリック済みかどうかを保存する名前
const firstKey = "moodle-first-clicked";
const secondKey = "moodle-second-clicked";

// ① Moodleトップページ
if (path === "/2026/") {
  const loginButton = document.querySelector(
    'a[href$="/login/index.php"]'
  );
  if (!loginButton) {
    console.log("ログインボタンなし");
    // ログイン済みなどでボタンがない場合は記録を初期化
    sessionStorage.removeItem(firstKey);
    sessionStorage.removeItem(secondKey);
  } else if (sessionStorage.getItem(firstKey) !== "yes") {
    sessionStorage.setItem(firstKey, "yes");
    console.log("最初のログインボタンをクリック");
    loginButton.click();
  } else {
    console.log("最初のログイン操作は実行済み");
  }
// ② 大学IDログイン画面
} else if (
  path === "/2026/login/index.php" ||
  path === "/2026/login/index_form.html"
) {
  const universityButton = document.querySelector(
    'a[href$="/auth/oidc/"]'
  );
  if (!universityButton) {
    console.log("大学IDログインボタンが見つかりません");
  } else if (sessionStorage.getItem(secondKey) !== "yes") {
    sessionStorage.setItem(secondKey, "yes");
    console.log("大学IDログインボタンをクリック");
    universityButton.click();
  } else {
    console.log("大学IDログイン操作は実行済み");
  }
} else {
  console.log("自動ログインの対象外ページです");
}
```

p74〜77｜新しい構文： `!loginButton` は要素がない、`getItem()` は実行済みの読み出し、`||` は「または」、`removeItem()` は保存した情報の削除です。**ログインボタンがない＝必ずログイン成功、ではありません。**

p78〜79｜実行済み判定・理解度チェック　Chrome

保存 → 更新 → Moodleで確認。もしログインが止まって再試行したいときは、Consoleから次のコードで記録を初期化します。

参照ファイル： `snippets/05-p078-clear-session.js`

```
// スライド78：動作確認の前にクリック記録を初期化する
sessionStorage.removeItem("moodle-first-clicked");
sessionStorage.removeItem("moodle-second-clicked");
```

p79の答え：`setItem()` ＝保存、`!loginButton` ＝要素が存在しない、`||` ＝または。

復帰ポイント： `p73時点の完成コード` 。この簡易方式は誤動作の繰り返し防止が目的で、すべてのエラーの自動復旧ではありません。

第6部　設定画面を作って完成させよう

対応：スライド p80〜p107　｜　**ここまでの目標：** 拡張機能から自動ログインのオン・オフを切り替える

p80〜83　これまでの `manifest.json` + `content.js` に、新しく `popup.html` と `popup.js` を追加して4ファイルにします。HTMLが画面、JavaScriptが動作を担当します。

p84〜88｜popup.htmlを作る　VS Code → 新規作成

設定画面のチェックボックスと表示用のHTMLです。スライドp85の内容に沿っています。

参照ファイル： `code/06-p093-popup-ready/popup.html`

```
<!DOCTYPE html>
<html lang="ja">
<head>
  <meta charset="UTF-8">
  <title>Moodle Auto Login</title>
  <style>
    body {
      width: 250px;
      padding: 15px;
      font-family: sans-serif;
    }
    h2 {
      font-size: 18px;
    }
    label {
      cursor: pointer;
    }
  </style>
</head>
<body>
  <h2>Moodle Auto Login</h2>
  <label>
    <input type="checkbox" id="enabled" disabled>
    自動ログインを有効にする
  </label>
  <p id="status">設定を読み込み中...</p>
  <script src="popup.js"></script>
</body>
</html>
```

p86 `input type="checkbox"` はチェックボックス、`id` は識別子、`disabled` は設定読み込み中の操作禁止。
p87 `<style>` は見た目の指定。**p88** `<script src="popup.js">` がJavaScriptを読み込みます。

p89〜91｜manifest.jsonを変更する　VS Code → 上書き

`storage` 権限と `action.default_popup` を追加し、`all_frames: true` も指定します。

参照ファイル： `code/06-p093-popup-ready/manifest.json`

```
{
  "manifest_version": 3,
  "name": "Moodle Auto Login",
  "version": "1.0.0",
  "description": "和歌山大学Moodleのログイン操作を自動化する拡張機能",
  "permissions": [
    "storage"
  ],
  "action": {
    "default_popup": "popup.html"
  },
  "content_scripts": [
    {
      "matches": [
        "https://moodle2026.wakayama-u.ac.jp/2026/*"
      ],
      "js": [
        "content.js"
      ],
      "run_at": "document_idle",
      "all_frames": true
    }
  ]
}
```

`all_frames: true` は **matchesに該当するフレームだけ**を対象とします。Microsoft認証ページのスクリプト実行権を与える指定ではありません。

p92〜93｜popup.jsを作る　VS Code → 新規作成

チェックの状態を読み、変更時に `chrome.storage.local` へ保存します。

参照ファイル： `code/06-p093-popup-ready/popup.js`

```
// HTMLの要素を取得
const checkbox = document.querySelector("#enabled");
const statusText = document.querySelector("#status");

// 設定状態を画面に表示する
function showStatus(enabled) {
  if (enabled) {
    statusText.textContent = "自動ログイン：ON";
  } else {
    statusText.textContent = "自動ログイン：OFF";
  }
}

// 保存されている設定を読み込む
async function loadSettings() {
  const data = await chrome.storage.local.get({ enabled: true });
  checkbox.checked = data.enabled;
  checkbox.disabled = false;
  showStatus(data.enabled);
}

// チェックボックスが変更されたとき
checkbox.addEventListener("change", async () => {
  const enabled = checkbox.checked;
  await chrome.storage.local.set({ enabled: enabled });
  showStatus(enabled);
});

// 最初に設定を読み込む
loadSettings();
```

p94〜99｜イベントと設定保存を理解する　コードを読む・Consoleで確認

```
checkbox.addEventListener("change", () => {
  console.log("設定が変更されました");
});
```

p94の `addEventListener()` は「チェック状態が変化したら処理を実行する」指定です。`popup.js` にすでに書いてあるので、上のコードを**重複して貼り付ける必要はありません**。

```
await chrome.storage.local.set({ enabled: true });
const data = await chrome.storage.local.get({ enabled: true });
```

`set` は保存、`get` は読み出し。`async/await` は結果を待つ書き方です。`sessionStorage` は一時的なWebページ側の記録、`chrome.storage.local` は拡張機能の設定を保持します。

p100〜104｜content.jsに設定判定を組み込む　VS Code → 上書き

参照ファイル： `code/06-p102-complete/content.js`（完成コード全文）

p101：前半（このあとにp102の後半を**同じファイルへ続けて書きます。**途中の状態では動きません）

```
async function autoLogin() {
  // ① 設定を読み込む
  const data = await chrome.storage.local.get({ enabled: true });

  // OFFなら何もしない
  if (!data.enabled) {
    console.log("自動ログインはOFFです");
    return;
  }

  console.log("Moodle Auto Login 起動！");
  const path = window.location.pathname;
  const firstKey = "moodle-first-clicked";
  const secondKey = "moodle-second-clicked";

  // ② Moodleトップページ
  if (path === "/2026/") {
    const loginButton = document.querySelector(
      'a[href$="/login/index.php"]'
    );

    // ボタンがなければ記録を初期化
    if (!loginButton) {
      sessionStorage.removeItem(firstKey);
      sessionStorage.removeItem(secondKey);
      return;
    }

    // 同じタブでクリック済みなら停止
    if (sessionStorage.getItem(firstKey) === "yes") {
      console.log("最初のログイン操作は実行済み");
      return;
    }

    sessionStorage.setItem(firstKey, "yes");
    loginButton.click();
```

p102：後半（前半の直後に貼り付けて、`content.js` 全体を完成させてください）

```
  // ③ 大学IDログイン画面
  } else if (
    path === "/2026/login/index.php" ||
    path === "/2026/login/index_form.html"
  ) {
    const universityButton = document.querySelector(
      'a[href$="/auth/oidc/"]'
    );
    if (!universityButton) {
      console.log("大学IDログインボタンがありません");
      return;
    }
    if (sessionStorage.getItem(secondKey) === "yes") {
      console.log("大学IDログイン操作は実行済み");
      return;
    }
    sessionStorage.setItem(secondKey, "yes");
    universityButton.click();
  }
}

// 自動ログインを開始
autoLogin();
```

p103〜104のポイント：最初に `chrome.storage.local.get({enabled:true})` で設定を読み、`if (!data.enabled) { return; }` でOFFのとき処理を終了します。

p105｜完成版を動作確認する　Chrome

①4ファイルを保存 → ②`chrome://extensions/` で更新 → ③右上の拡張機能アイコンを押す → ④チェックをOFFにしてMoodleを新しいタブで開く → ⑤ONに戻して動作確認。設定を変えただけでは開いているページのコードがその場で再実行されるとは限りません。

p106〜107｜最終理解度チェック・まとめ　理解度チェック

p106の答え：①`popup.html`＝設定画面を作る、②`chrome.storage.local.set()`＝設定を保存する、③OFFのとき＝`return` で自動ログイン処理を終了。

復帰ポイント： `p93時点の4ファイル` / `p102 最終完成版` 。

全スライド対応インデックス（p1〜107）

以下はPDFの**スライド番号を変えずに**、何をするページなのかを一行でまとめた索引です。説明を聞くページも含め、全107ページを掲載しています。

導入：p1〜p5

スライド	内容・確認ポイント
p001	タイトル・導入

スライド	内容・確認ポイント
p002	完成デモ
p003	問題意識
p004	Chrome/VS Codeの準備
p005	認証フロー

第1部：p6〜p21

スライド	内容・確認ポイント
p006	第1部開始
p007	１Chrome 拡張機能の構造を知る
p008	１Chrome 拡張機能の構造を知る
p009	フォルダー作成（コマンド）
p010	2ファイル作成
p011	STEP 2. manifest.jsonを作る
p012	manifest.json（新規）
p013	ここを理解しておこう13
p014	ここを理解しておこう少し解説します.
p015	理解しておこう例えば、
p016	content.js（新規）
p017	STEP 3. content.jsを作る
p018	Chromeデベロッパーモード
p019	拡張機能読み込み
p020	Consoleで確認
p021	確認問題

第2部：p22〜p33

スライド	内容・確認ポイント
p022	第2部開始
p023	HTMLのaタグ
p024	STEP 1. まずはHTML を理解しようそれぞれの意味を確認しよう！
p025	MoodleのリンクHTML
p026	「検証」で探す

スライド	内容・確認ポイント
p027	ConsoleでquerySelector
p028	STEP 3. JavaScriptでボタンを取得する
p029	Consoleでclick
p030	STEP 5. ボタンが見つからなかったら？ただし，1つ問題があります．
p031	null対策のif
p032	STEP 5. ボタンが見つからなかったら？if は条件分岐です．
p033	理解度チェック

第3部：p34〜p46

スライド	内容・確認ポイント
p034	第3部開始
p035	ログインボタンを自動クリックしよう自動化には成功しました．
p036	content.js上書き
p037	STEP 2. content.jsを書き換える
p038	ここでコードを理解しよう38
p039	拡張機能更新
p040	STEP 3. 拡張機能を更新するただし，すでにログイン済みの場合はボタンが存在せず
p041	STEP 4. 少しだけ改良してみよう現在のコードはMoodle 内のいろいろなページ
p042	pathnameを表示
p043	content.js条件分岐
p044	STEP 5. 条件分岐を追加する44
p045	Consoleログ
p046	理解度チェック

第4部：p47〜p60

スライド	内容・確認ポイント
p047	第4部開始
p048	STEP 1. まず、何が足りないのか考えよう48
p049	大学IDのリンクHTML
p050	Consoleで取得
p051	Consoleでクリック

スライド	内容・確認ポイント
p052	STEP 4. ページごとに処理を分ける今回の拡張機能では，2つの異なるページでボタン
p053	else ifサンプル
p054	if とelse if の違いこのコードを実行すると
p055	content.jsを2画面対応
p056	コードを分解して理解しよう①現在のパスを取得する
p057	コードを分解して理解しよう③2つ目のログイン画面なら大学
p058	手順の動作確認
p059	STEP 6. 実際に動作確認してみよう59
p060	応用問題

第5部：p61〜p79

スライド	内容・確認ポイント
p061	第5部開始
p062	STEP 1. まず、問題が起こる場面を考えよう今までのコードは，ボタンを見つけるとす
p063	STEP 1. まず、問題が起こる場面を考えよう63
p064	STEP 1. まず、問題が起こる場面を考えよう認証がうまくいかないと
p065	sessionStorage保存
p066	sessionStorage読み出し
p067	sessionStorage削除
p068	3つの命令を整理しようsessionStorage のデータは，同じタブでページを再読
p069	実行済み判定
p070	STEP 3. 一度だけクリックする仕組みを考えよう最初の実行では，
p071	おまけでは，どうして保存してからクリックするのでしょう
p072	STEP 4. 前回のコードを改良しようそれでは，VS Codeでcontent.js
p073	content.js上書き
p074	STEP 5. 追加した部分を理解しよう①! は「ではない」
p075	STEP 5. 追加した部分を理解しよう②sessionStorage
p076	STEP 5. 追加した部分を理解しよう③／／ は「または」
p077	STEP 5. 追加した部分を理解しよう④記録を初期化する
p078	再実行防止の動作確認
p079	理解度チェック

第6部：p80〜p107

スライド	内容・確認ポイント
p080	第6部開始
p081	STEP 1. 今回作るものを見てみよう最終的には，拡張機能のアイコンをクリックすると
p082	4ファイル構成
p083	STEP 2. ファイルを追加しようそれぞれの役割を整理すると，以下のようになります
p084	STEP 3. popup.htmlを作ろう
p085	popup.html新規
p086	STEP 3. popup.htmlを作ろう
p087	STEP 3. popup.htmlを作ろう
p088	STEP 3. popup.htmlを作ろう
p089	STEP 4. manifest.jsonを変更しよう
p090	manifest.json更新
p091	STEP 4. manifest.jsonを変更しよう
p092	STEP 5. popup.jsを作ろう
p093	popup.js新規
p094	changeイベント
p095	STEP 5. popup.jsを作ろう
p096	STEP 5. popup.jsを作ろう
p097	chrome.storageのset/get
p098	STEP 5. popup.jsを作ろう
p099	保存先の違い
p100	STEP 6. content.jsも設定に対応させよう
p101	content.js前半
p102	content.js後半
p103	STEP 6. content.jsも設定に対応させよう
p104	STEP 6. content.jsも設定に対応させよう
p105	最終動作確認
p106	理解度チェック
p107	おわり

途中で困ったときの確認表

症状	最初に確認すること
拡張機能を読み込めない	`manifest.json` が正しいJSONか、ファイル名が `.json.txt` ではないか
Consoleに出ない	`matches` がMoodleのURLと一致するか、拡張機能・ページを再読み込みしたか
`null` と表示される	ログイン済みではないか、正しいページで実行しているか
p55の2つ目が動かない	実際のパスが `index.php` と `index_form.html` のどちらか
同じ画面から再実行できない	p78の `sessionStorage` 削除コードで実行済み記録を消す
チェックを切り替えても動かない	変更後にMoodleを新しいタブで開いたか
途中から復帰したい	スライド番号 → `code/` の対応フォルダー → ファイルを比較

毎回の基本サイクル： `Ctrl + S` → Chrome拡張機能の「更新」 → Moodleのページ再読み込み → `F12` のConsoleで確認。**複数の完成版を同時にONにしないでください。**

2日開催の考え方：特定の日付・時間で章を区切りません。各章の「復帰ポイント」で作業を保存し、次回はそこから開始できます。進度が速い人は自分で次の章に進めます。

出典：AppLii「Moodle の拡張機能を作ってみよう！」最新版スライド（107ページ）と、同スライドに対応したGitHub用ソースコード。パスやコードは基本的にスライドに合わせています。p55のURL差異は教材内の確認事項として明記しています。